

Python Fundamentals

Writing Python Code (Notepad vs IDE)

- Save files with meaningful names, no spaces.
- End with .py extension.
- Case sensitivity matters.
- Run via cmd → python filename.py.
- IDEs (Integrated Development Environments) simplify coding, testing, debugging with smart suggestions and project organization.
- Notepad = manual effort; IDE = efficient coding.

Python Syntax & Structure

- Rules: Simple, straightforward.
- Indentation (spaces) defines blocks.
- Wrong indentation → error.
- Programs execute top → bottom, line by line.

Basic Input & Output

- Input waits for user response before continuing.
- Example run: user types → stored in variable → displayed with print
- ```
msg = input("What is your favorite Programming Language?")
print(msg)
```

## Concatenation (Joining Strings)

- Use + operator to join strings.  
Example:  

```
print("Hello " + input("What is your name?"))
```
- Input captures user response, print combines with "Hello ".

## Code Comments & Documentation

- Purpose: add notes/explanations for clarity.
- Single line: (hash)  
    # This is a comment
- Multi-line: (triple quote)  
    """  
    **Multi-line comment**  
    """
- Comments are ignored during execution.

## Errors & Debugging

- Variables = containers for values.
- Example
  - `x = 5`      # integer
  - `z = "John"` # string
- Declaring = creating a name.
- Initializing = assigning a value.
- Python auto-detects data type (int, string, float).
- Variables make code reusable & easy to update.
- Example benefit: change "apple" → "orange" once in variable, not everywhere in code.

## Constants

- Constants = values that don't change.
- Example:
  - `PI = 3.14`
  - `radius = 5`
  - `area = PI * radius * radius`
- Difference:
  - Variable → value changes.
  - Constant → fixed value.
- Benefits: clarity, reliability, prevents mistakes, easier debugging.
- Example output:
  - The area of the circle is: 78.5